Reviewer Pack — RSK Shape Concentration Separates Permanent from Padded Dete...

Entry c0170ef261da · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-18 17:28:13 UTC

RSK Shape Concentration Separates Permanent from Padded Determinant

Entry ID: `c0170ef261da` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-18 17:28:13 UTC

1. Conjecture

Field A (mathematical branch): Robinson-Schensted-Knuth combinatorics and Plancherel measure on Young diagrams (Schensted 1961; Logan-Shepp 1977; Vershik-Kerov 1977; Baik-Deift-Johansson 1999; Romik 2015 'The Surprising Mathematics of Longest Increasing Subsequences'). The RSK shape map σ ↦ λ(σ) ⊢ n is computed by patience-sort / row insertion — a comparison-and-bumping algorithm with NO polynomial extension over F_q, since longest-increasing-subsequence length is a max/sort functional that does not algebrize per Aaronson-Wigderson. Distinct from blacklisted Specht-isotype mass (which projects c(f) onto λ-isotypes via character sums Σ_σ χ^λ(σ)c_σ) — here we PARTITION S_n by RS shape and sum |c_σ|² fiberwise, never invoking χ^λ; distinct from Eulerian-descent entropy (a Worpitzky descent-count binning, not an LIS shape); distinct from Patience-Sort LIS XOR-lifted (different target object: XOR-lifted CC matrices on F_2^n, not the GCT permutation-coefficient vector of perm vs padded-det). An arXiv search 'RSK shape distribution' AND ('determinantal complexity' OR 'GCT' OR 'permanent versus determinant' OR 'orbit closure') returns 0 direct hits with <5 adjacent papers, all on Plancherel asymptotics in random-matrix theory. **Field B** (complexity object): Mulmuley-Sohoni Geometric Complexity Theory: determinantal complexity dc(perm_n), the smallest m such that perm_n ∈ GL_{n²}·orbit-closure of ℓ(y)^{n-m}·det_m(L(y)) for ℓ a linear form and L an m×m matrix of linear forms in the n² matrix variables y={y_{ij}} (Mignon-Ressayre 2004 n²/2 lower bound; Landsberg 2017; Burgisser-Ikenmeyer 2013). The permutation-coefficient vector c(f) ∈ Z^{n!} with c_σ(f) := coeff of ∏*i* x{i,σ(i)} in f gives a canonical S_n × S_n-equivariant structural fingerprint that is preserved under row/column-permutation substitutions (a subgroup of GL_{n²}), suitable as a partial orbit-closure obstruction in the canonical det-vs-perm separation testbed.

Statement:

> For a homogeneous polynomial f of degree n in matrix variables {x_{ij}}*{1≤i,j≤n}, let c_σ(f) := coeff of* ∏i x*{i,σ(i)} in f, define the RSK-shape weighted distribution P_f(λ) :=* (Σ̄{σ:λ(σ)=λ} c_σ(f)²)/(Σ_σ c_σ(f)²) when the denominator is nonzero, and set K(f) := max_{λ⊢n} P_f(λ). Conjecture: there exist absolute constants n_0, C such that for every n ≥ n_0 and every padded determinant g = ℓ^{n-m}·det_m(L) with m ≤ n-1, ℓ a nonzero linear form, L an m×m matrix of linear forms over Q in {x_{ij}}, and Σ_σ c_σ(g)² > 0, the inequality K(g) ≥ C·K(perm_n) holds with C = 2 and n_0 = 5. A single (n,m,ℓ,L) tuple at n ∈ {5,6,7,8} with K(g) < 2·K(perm_n) and Σ_σ c_σ(g)² > 0 falsifies the conjecture.

Rationale (proposer's reasoning):

The all-ones coefficient vector of perm_n drives P_{perm_n} to be EXACTLY the Plancherel measure (f^λ)²/n!, whose maximum at the Vershik-Kerov limit shape decays as O(n^{-1/4}) — perm_n is maximally RS-diffuse. Padded determinants, by contrast, have multilinear-permutation coefficients controlled by sgn(ρ)·signed-permanent-cofactor sums over only n!/(m!·(n-m)!)-many σ-supports, concentrating |c_σ(g)|² on permutations factoring through a fixed m-block — these factorings live in a small union of (P,Q)-shape fibers, forcing K(g) ≥ constant. The bridge therefore predicts a quantitative diffuseness gap that is structural on (□,L) (two padded forms with identical Boolean truth tables can have different K), shielding from Razborov-Rudich; and the RSK map uses only comparisons, shielding from Aaronson-Wigderson algebrization.

Taxonomy category: `GCT` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `87e6db3e38b07513`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each n∈{5,6,7,8} and m∈{1,2,3}, run 30 seeds generating padded determinants g with nonzero RSK-weight denominator; compute ratio R=K(g)/K(perm_n). SUPPORTED iff every seed yields R≥2 AND per-(n,m) median R≥2. FALSIFIED if any single valid seed gives R<2.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): `- RSK shape permutation coefficients determinantal complexity permanent - Plancherel measure Young diagram geometric complexity theory orbit closure - longest increasing subsequence permanent versus determinant padded obstruction`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```
import sys
import random
import math
import itertools
```

```
from fractions import Fraction

def matrix_mult(A, B):
    n = len(A)
    m = len(B[0])
    p = len(B)
    result = [[0 for _ in range(m)] for _ in range(n)]
    for i in range(n):
        for j in range(m):
            for k in range(p):
                result[i][j] += A[i][k] * B[k][j]
    return result

def matrix_det(M):
    n = len(M)
    if n == 1:
        return M[0][0]
    det = 0
    for col in range(n):
        minor = [row[:col] + row[col+1:] for row in M[1:]]
        det += ((-1) ** col) * M[0][col] * matrix_det(minor)
    return det

def compute_c_perm_n(n):
    return [1] * math.factorial(n)

def compute_rs_shape(sigma):
    piles = []
    for num in sigma:
        placed = False
        for pile in piles:
            if num > pile[-1]:
                pile.append(num)
                placed = True
                break
        if not placed:
            piles.append([num])
    return tuple(len(pile) for pile in piles)

def compute_K_perm_n(n):
    c_perm_n = compute_c_perm_n(n)
    shapes = {}
    for sigma in itertools.permutations(range(1, n+1)):
        shape = compute_rs_shape(sigma)
        shapes[shape] = shapes.get(shape, 0) + 1
    total = sum(c_perm_n)
    if total == 0:
        return 0
    max_prob = max(shapes.values()) / total
    return max_prob

def compute_c_padded_det(n, m, l, L, seed):
    random.seed(seed)
    c = [0] * math.factorial(n)
    x = [[random.randint(-2, 2) for _ in range(n)] for _ in range(n)]
    for sigma in itertools.permutations(range(n)):
```

```
        sigma_list = list(sigma)
        term = 1
        for i in range(n):
            term *= l[i] * x[i][sigma_list[i]]
        for rho in itertools.permutations(range(m)):
            for T in itertools.combinations(range(n), m):
                for pi in itertools.permutations(range(m)):
                    if len(set(pi)) != m:
                        continue
                    coef = 1
                    for i in range(m):
                        if i not in T:
                            continue
                        coef *= L[pi[i]][rho[pi[i]]]
                    term *= coef
        c[sigma] = term
    return c

def run_trial(seed):
    random.seed(seed)
    n = random.choice([5, 6, 7, 8])
    m = random.randint(1, 3)
    l = [random.randint(-2, 2) for _ in range(n)]
    L = [[random.randint(-1, 1) for _ in range(m)] for _ in range(m)]
    c_padded_det = compute_c_padded_det(n, m, l, L, seed)
    K_perm_n = compute_K_perm_n(n)
    shapes = {}
    total = sum(c_padded_det)
    if total == 0:
        return {
            "metric_name": "K(g)/K(perm_n)",
            "metric_value": 0,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "denominator_zero"
        }
    for sigma in itertools.permutations(range(n)):
        shape = compute_rs_shape(list(sigma))
        shapes[shape] = shapes.get(shape, 0) + c_padded_det[sigma] ** 2
    max_prob = max(shapes.values()) / total
    ratio = max_prob / K_perm_n
    conjecture_holds = ratio >= 2
    counterexample = "" if conjecture_holds else f"n={n}, m={m}, ratio={ratio}"
    return {
        "metric_name": "K(g)/K(perm_n)",
        "metric_value": ratio,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17,
    trials = []
    for seed in seeds:
        trial = run_trial(seed)
```

```
        trials.append(trial)
        print(f"TRIAL: {trial}")
    metric_values = [trial["metric_value"] for trial in trials]
    mean = sum(metric_values) / len(metric_values)
    std = math.sqrt(sum((x - mean) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = sum(1 for trial in trials if trial["conjecture_holds"]) / len(trials)
    if all(trial["conjecture_holds"] for trial in trials):
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    else:
        for trial in trials:
            if not trial["conjecture_holds"]:
                print(f"RESULT: FALSIFIED counterexample={trial['counterexample']} first_failing_s
                break
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_613d9fe5.py", line 126, in <module>
    trial = run_trial(seed)
            ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_613d9fe5.py", line 95, in run_trial
    c_padded_det = compute_c_padded_det(n, m, l, L, seed)
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_613d9fe5.py", line 86, in compute_c_pad
    c[sigma] = term
    ~^^^^^^^
TypeError: list indices must be integers or slices, not tuple
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed with a TypeError (list indexed by tuple) in compute_c_padded_det before any seeds produced data, so neither the support nor falsification condition could be evaluated. | next: Fix the coefficient storage to use a dict keyed by permutation tuples (or convert sigma to a flat index) and rerun the pre-registered 30-seed sweep over (n,m) ∈ {5,6,7,8}×{1,2,3}.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	188904
2	preregistration	claude_max	opus	0	0	8702
3	novelty	claude_max	opus	0	0	4123
4	novelty	claude_max	opus	0	0	7123
5	test_gen	mistral	codestral-latest	0	0	376234
6	test_gen	mistral	codestral-latest	0	0	314331
7	test_gen	mistral	codestral-latest	0	0	315597
8	test_gen	mistral	codestral-latest	0	0	313045
9	judge	claude_max	opus	0	0	4484

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 1532544 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 4}

(full prompt+response transcripts available in `research/audit/c0170ef261da.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c0170ef261da.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c0170ef261da.tar.gz` (if generated)